МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РЕСПУБЛИКИ КАЗАХСТАН

АЛМАТИНСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ
Кафедра информационных систем

ОТЧЁТ

по лабораторной работе №11

Проектирование стратегии шардирования и тест распределения

Дисциплина: «Проектирование и применение NoSQL-технологий»
3 курс • 2026–2027 учебный год
Вариант 09: IoT-датчики

Выполнил: студент Анваров Акрам

Алматы, 2026

1. Цель работы, вариант и предметная область

Цель: спроектировать стратегию горизонтального масштабирования документного хранилища, выбрать shard key на основе Access Patterns, количественно проверить распределение документов и нагрузки, выявить горячие шарды и обосновать архитектурное решение.

Вариант 09 — IoT-датчики. Кандидатный ключ: `deviceId`; альтернативный низкокардинальный ключ: `deviceType`; основной запрос — показания устройства. **Предметная область:** платформа мониторинга, в которой датчики разных типов (температура, влажность, давление, вибрация, освещённость) непрерывно отправляют показания. Коллекция `readings` содержит поля `deviceId` (2500 уникальных устройств), `deviceType` (5 типов), `readingId` (монотонно растущий номер показания), `dayBucket` (номер суток, используется в шаге 4). В модели: N = 12 000 документов, Q = 30 000 запросов, 3 шарда (A, B, C). Все данные синтетические.

Соответствие переименования из методички: `user_id → deviceId`, `category → deviceType`, `event_id → readingId`.

2. Access Patterns

ID	Описание	Тип	Частота	Фильтр / сортировка
AP-01	Последние показания конкретного устройства (основной запрос)	чтение	45%	deviceId = X, sort readingId desc, limit 20
AP-02	Показания всего парка за интервал времени	чтение	10%	readingId (время) в диапазоне [a, b]
AP-03	Агрегация показаний по типу устройства (средние, max)	агрегация	8%	group by deviceType
AP-04	Загрузка нового потока показаний	запись	30%	insertMany, новые readingId
AP-05	Показания устройства за интервал	чтение	5%	deviceId = X и readingId/dayBucket в диапазоне
AP-06	Список устройств с тревожными показаниями за последний час	чтение	2%	alert = true, время ≥ now − 1h

Вывод: доминируют запросы по одному устройству (AP-01, AP-05 — 50%) и запись (AP-04 — 30%). Ключ должен одновременно давать целевую маршрутизацию по `deviceId` и распределять вставки.

3. Кандидатные shard key

Ключ	Кардинальность	Частота значения	Монотонность	Оценка
A: deviceType	5 (низкая)	≈20% документов на значение	нет	Не более 5 «кусков» данных; при 3 шардах размещение 2/2/1; один тип легко становится горячим
B: deviceId (hashed)	2500 (достаточная, ≥1000)	≈4,8 документа на значение (0,04%)	нет (идентификатор не зависит от времени)	Хороший баланс и targeted для AP-01/AP-05; риск — одно очень «шумное» устройство
C: readingId (ranged)	12 000 (уникален)	1 документ на значение	ДА, строго возрастает	Идеален для диапазонов (AP-02), но все новые вставки идут в последний диапазон (hot shard)

4. Исходный код, инструкция запуска, seed

Требования: Python 3.10+ и стандартная библиотека (дополнительных пакетов нет). Запуск: `python simulation.py` и `python simulation_step4.py`. Seed зафиксирован: `random.seed(42)`. Скрипт запускался дважды — результаты полностью совпали (рис. 2–3). Хеш SHA-256 используется только для учебной симуляции и не воспроизводит внутренний алгоритм MongoDB.

Листинг 1. simulation.py (основной эксперимент, шаги 1–2)

```
# Лабораторная работа №11, вариант 09 (IoT-датчики)
# candidate: deviceId, alternative low-cardinality key: deviceType
import random, hashlib
from collections import Counter
random.seed(42)
N, Q, SHARDS = 12000, 30000, 3
rows = [dict(deviceId=i % 2500,
             deviceType=i % 5,
             readingId=i) for i in range(N)]

def stable_hash(value):
    digest = hashlib.sha256(str(value).encode()).digest()
    return int.from_bytes(digest[:8], 'big')

def low_card(row):
    return row['deviceType'] % SHARDS

def hashed_device(row):
    return stable_hash(row['deviceId']) % SHARDS

def range_reading(row):
    return min(row['readingId'] // (N // SHARDS), SHARDS - 1)

def describe(name, assign, requests):
    docs = Counter(assign(r) for r in rows)
    load = Counter(assign(r) for r in requests)
    d = [docs[i] for i in range(SHARDS)]
    q = [load[i] for i in range(SHARDS)]
    print(name, 'docs=', d, 'queries=', q,
          'doc_max_mean=', round(max(d)/(sum(d)/SHARDS), 2),
          'query_max_mean=', round(max(q)/(sum(q)/SHARDS), 2))

uniform = random.choices(rows, k=Q)
hot_rows = [r for r in rows if r['deviceType'] == 0]
hot = [random.choice(hot_rows)
       if random.random() < 0.6 else random.choice(rows)
       for _ in range(Q)]

for name, fn in [('deviceType', low_card),
                 ('hashed_device', hashed_device),
                 ('range_reading', range_reading)]:
    describe(name + '/uniform', fn, uniform)
    describe(name + '/hot', fn, hot)
```

Листинг 2. simulation_step4.py (шаг 4: перекос 80%, альтернативный ключ, горячее устройство, поток вставок)

```
# Шаг 4: перекос 80% в одной категории + альтернативный ключ deviceId + dayBucket
# + доп. эксперимент "горячее устройство"
import random, hashlib
from collections import Counter
random.seed(42)
N, Q, SHARDS = 12000, 30000, 3

def stable_hash(value):
    digest = hashlib.sha256(str(value).encode()).digest()
    return int.from_bytes(digest[:8], 'big')

# 80% записей относятся к deviceType = 0
rows = []
for i in range(N):
    t = 0 if random.random() < 0.8 else random.randint(1, 4)
    rows.append(dict(deviceId=i % 2500, deviceType=t,
                     readingId=i, dayBucket=i // 2500))

def low_card(r):      return r['deviceType'] % SHARDS
def hashed_device(r): return stable_hash(r['deviceId']) % SHARDS
def range_reading(r): return min(r['readingId'] // (N // SHARDS), SHARDS - 1)
def hashed_dev_day(r):
    return stable_hash((r['deviceId'], r['dayBucket'])) % SHARDS

def describe(name, assign, requests):
    docs = Counter(assign(r) for r in rows)
    load = Counter(assign(r) for r in requests)
    d = [docs[i] for i in range(SHARDS)]
    q = [load[i] for i in range(SHARDS)]
    print(f'{name:34s} docs={d} queries={q} '
          f'doc_max_mean={max(d)/(sum(d)/SHARDS):.2f} '
          f'query_max_mean={max(q)/(sum(q)/SHARDS):.2f}')

share0 = sum(r['deviceType'] == 0 for r in rows) / N
print('доля deviceType=0:', round(share0 * 100, 1), '%')

uniform = random.choices(rows, k=Q)
hot_rows = [r for r in rows if r['deviceType'] == 0]
hot_cat = [random.choice(hot_rows) if random.random() < 0.6
           else random.choice(rows) for _ in range(Q)]
# горячее устройство: 60% запросов к deviceId = 7
dev_rows = [r for r in rows if r['deviceId'] == 7]
hot_dev = [random.choice(dev_rows) if random.random() < 0.6
           else random.choice(rows) for _ in range(Q)]

keys = [('deviceType', low_card), ('hashed_device', hashed_device),
        ('range_reading', range_reading),
        ('hashed(device+day)', hashed_dev_day)]
print('--- набор данных 80% ---')
for name, fn in keys:
    describe(name + '/uniform', fn, uniform)
    describe(name + '/hot-category', fn, hot_cat)
print('--- горячее устройство (60% к deviceId=7) ---')
for name, fn in keys:
    describe(name + '/hot-device', fn, hot_dev)

print('--- кардинальность ---')
print('deviceType      :', len({r['deviceType'] for r in rows}))
print('deviceId        :', len({r['deviceId'] for r in rows}))
print('readingId       :', len({r['readingId'] for r in rows}))
print('device+dayBucket:', len({(r['deviceId'], r['dayBucket']) for r in rows}))

# AP-04: загрузка нового потока (3000 новых показаний, новый день dayBucket=5)
print('--- загрузка нового потока: 3000 записей (куда попадут вставки) ---')
new = [dict(deviceId=random.randrange(2500),
            deviceType=0 if random.random() < 0.8 else random.randint(1, 4),
            readingId=N + j, dayBucket=5) for j in range(3000)]
def range_new(r):  # границы диапазонов зафиксированы при начальной загрузке
    return min(r['readingId'] // (N // SHARDS), SHARDS - 1)
for name, fn in [('deviceType', low_card), ('hashed_device', hashed_device),
                 ('range_reading', range_new), ('hashed(device+day)', hashed_dev_day)]:
    c = Counter(fn(r) for r in new)
    print(f'{name:22s} inserts={[c[i] for i in range(SHARDS)]}')
```

5. Результаты эксперимента (шаги 1–2)

Показатели: доля документов = count_shard / N × 100%; доля запросов = requests_shard / Q × 100%; max/mean = max(counts) / mean(counts). Режим *uniform* — запросы равномерно по документам; режим *hot* — 60% запросов к документам одной категории (`deviceType = 0`).

Ключ / нагрузка	Документы A/B/C	Запросы A/B/C	Max/mean (запросы)
deviceType / uniform	4800 / 4800 / 2400	11947 / 12018 / 6035	1,20
deviceType / hot	4800 / 4800 / 2400	22828 / 4822 / 2350	**2,28**
hashed_device / uniform	3866 / 3805 / 4329	9608 / 9435 / 10957	1,10
hashed_device / hot	3866 / 3805 / 4329	9261 / 9774 / 10965	1,10
range_reading / uniform	4000 / 4000 / 4000	10046 / 9958 / 9996	1,00
range_reading / hot	4000 / 4000 / 4000	9967 / 9985 / 10048	1,00

Таблица 1. Распределение документов и запросов по шардам (доли документов: deviceType — 40/40/20%; hashed — 32,2/31,7/36,1%; range — 33,3% на шард).

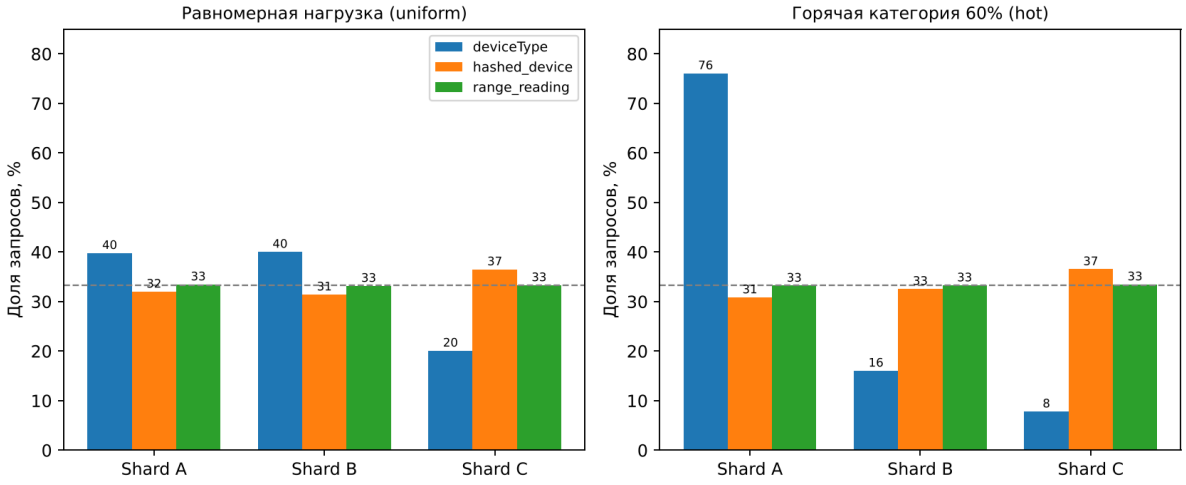


Рисунок 1. Столбчатый график распределения нагрузки по шардам (доля запросов, %). Пунктир — идеальные 33,3%.

Почему равномерное число документов не означает равномерной нагрузки. У ключа deviceType документы распределены 40/40/20%, но в режиме hot шард A получает 22 828 из 30 000 запросов (76,1%). Это ожидаемо: 60% запросов идут к типу 0 (шард A), а оставшиеся 40% распределяются как 40/40/20, т. е. 0,6 + 0,4 × 0,4 = 0,76. Объём данных определяется тем, *где лежат документы*, а нагрузка — тем, *куда обращаются запросы*; популярность значений ключа (skew) влияет только на второе. Ключи deviceId и readingId не коррелируют с типом устройства, поэтому «горячая категория» размазывается по всем шардам (max/mean 1,10 и 1,00). Это не значит, что они всегда устойчивы к перекосу — см. эксперимент «горячее устройство» в п. 7.

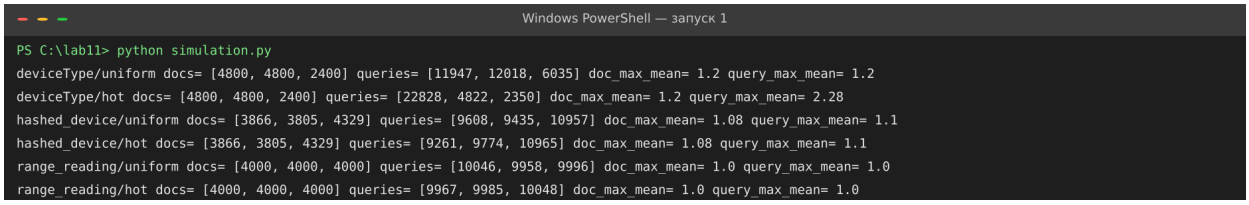


Рисунок 2. Скриншот: запуск 1 simulation.py (выводы совпадают с табл. 1).

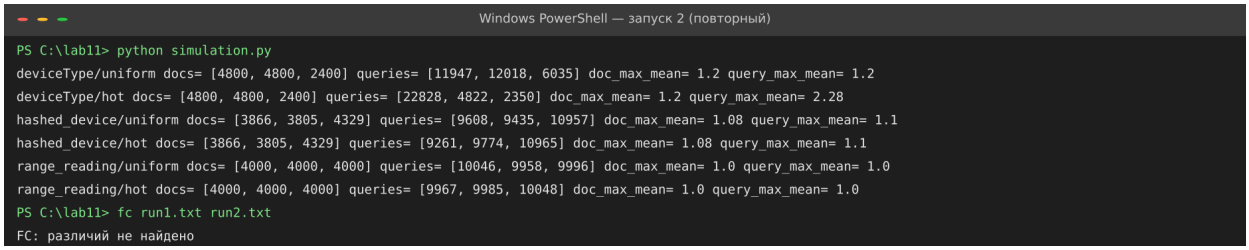


Рисунок 3. Скриншот: повторный запуск simulation.py и сравнение выводов (различий нет — воспроизводимость подтверждена).

6. Архитектура и targeted / scatter-gather

Приложение обращается не к отдельным шардам, а к маршрутизатору **mongos**. Если в фильтре есть shard key (или его префикс), mongos направляет запрос на один шард — **targeted**. Если ключа в фильтре нет, запрос рассылается на все шарды, а результаты собираются и объединяются —

scatter-gather; он дороже по задержке и нагрузке. Репликация (копии одних и тех же данных ради доступности) шардирование не заменяет: шард обычно сам является replica set.

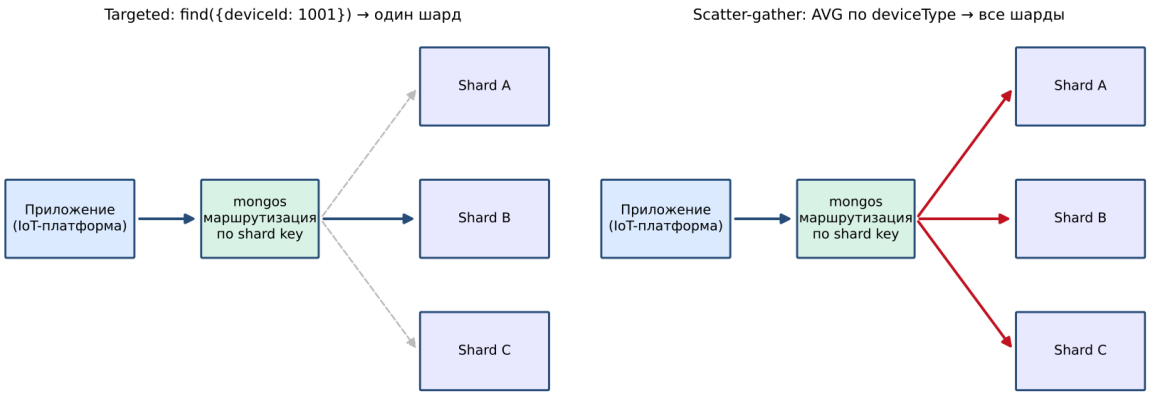


Рисунок 4. Маршрутизация запросов через mongos: targeted (слева) и scatter-gather (справа). Упрощённая схема, не реальный mongos.

AP	A: deviceType	B: hashed deviceId	C: ranged readingId	Alt: deviceId + dayBucket
AP-01 по устройству	scatter-gather	**targeted**	scatter-gather	targeted при указании dayBucket; иначе несколько шардов
AP-02 интервал по парку	scatter-gather	scatter-gather	**targeted** (1–2 шарда)	scatter-gather
AP-03 агрегат по типу	**targeted** (1 шард)	scatter-gather	scatter-gather	scatter-gather
AP-04 запись потока	один шард на тип (перекос)	**равномерно**	всё в последний шард (hot shard)	**равномерно**
AP-05 устройство + интервал	scatter-gather	**targeted**	scatter-gather	**targeted** (оба поля)
AP-06 тревоги за час	scatter-gather	scatter-gather	scatter-gather / частично targeted	scatter-gather

Таблица 2. Маршрутизация Access Patterns для каждого ключа (в учебной симуляции распределение по модулю — упрощение).

7. Улучшение модели (шаг 4)

Набор данных изменён так, что 80,1% записей относятся к `deviceType = 0`. Предложен альтернативный ключ `deviceId + dayBucket` (кардинальность 12 000 пар — в 4,8 раза выше, чем у одного deviceId; для эффективного targeted-запроса в фильтре нужны **оба** поля, иначе запрос затронет несколько шардов). Дополнительно проверен сценарий «горячее устройство» (60% запросов к `deviceId = 7`) и распределение вставок нового потока (AP-04).

Ключ / нагрузка	Документы A/B/C	Запросы A/B/C	Max/mean (запросы)
deviceType / uniform (80%)	10213 / 1200 / 587	25515 / 2987 / 1498	**2,55**
deviceType / hot-category	10213 / 1200 / 587	28251 / 1163 / 586	**2,83**
hashed_device / uniform	3866 / 3805 / 4329	9803 / 9533 / 10664	1,07
hashed_device / hot-category	3866 / 3805 / 4329	9702 / 9480 / 10818	1,08
range_reading / uniform	4000 / 4000 / 4000	9894 / 10077 / 10029	1,01
range_reading / hot-category	4000 / 4000 / 4000	10040 / 9927 / 10033	1,00
hashed(device+day) / uniform	3995 / 4000 / 4005	9939 / 9965 / 10096	1,01
hashed(device+day) / hot-category	3995 / 4000 / 4005	9938 / 9984 / 10078	1,01
deviceType / hot-device	10213 / 1200 / 587	24601 / 4793 / 606	2,46
hashed_device / hot-device	3866 / 3805 / 4329	3900 / 3702 / 22398	**2,24**
range_reading / hot-device	4000 / 4000 / 4000	11152 / 11274 / 7574	1,13
hashed(device+day) / hot-device	3995 / 4000 / 4005	7624 / 14808 / 7568	**1,48**

Таблица 3. Результаты на наборе с перекосом 80% и в сценарии «горячее устройство».

Ключ	Вставки 3000 новых записей, A/B/C	Вывод
deviceType	2523 / 323 / 154	перекос по популярному типу (84%)
hashed_device	933 / 958 / 1109	равномерно
range_reading (монотонный)	**0 / 0 / 3000**	весь поток в один шард — hot shard
hashed(device+day)	1043 / 933 / 1024	равномерно

Таблица 4. Куда попадает новый поток (AP-04).

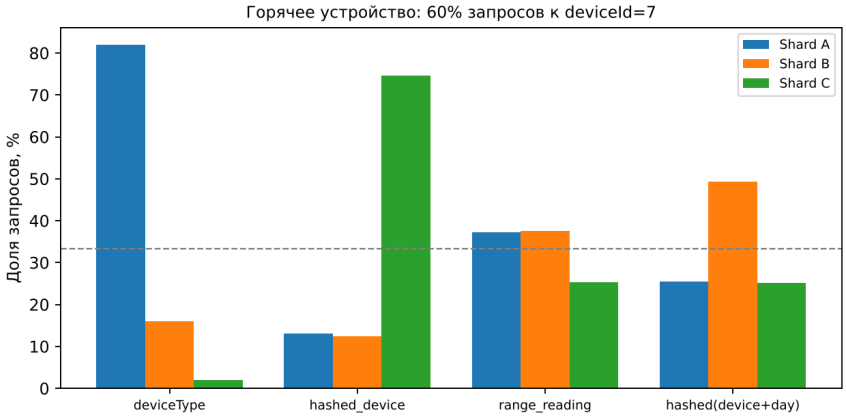


Рисунок 5. Доля запросов по шардам в сценарии «горячее устройство».

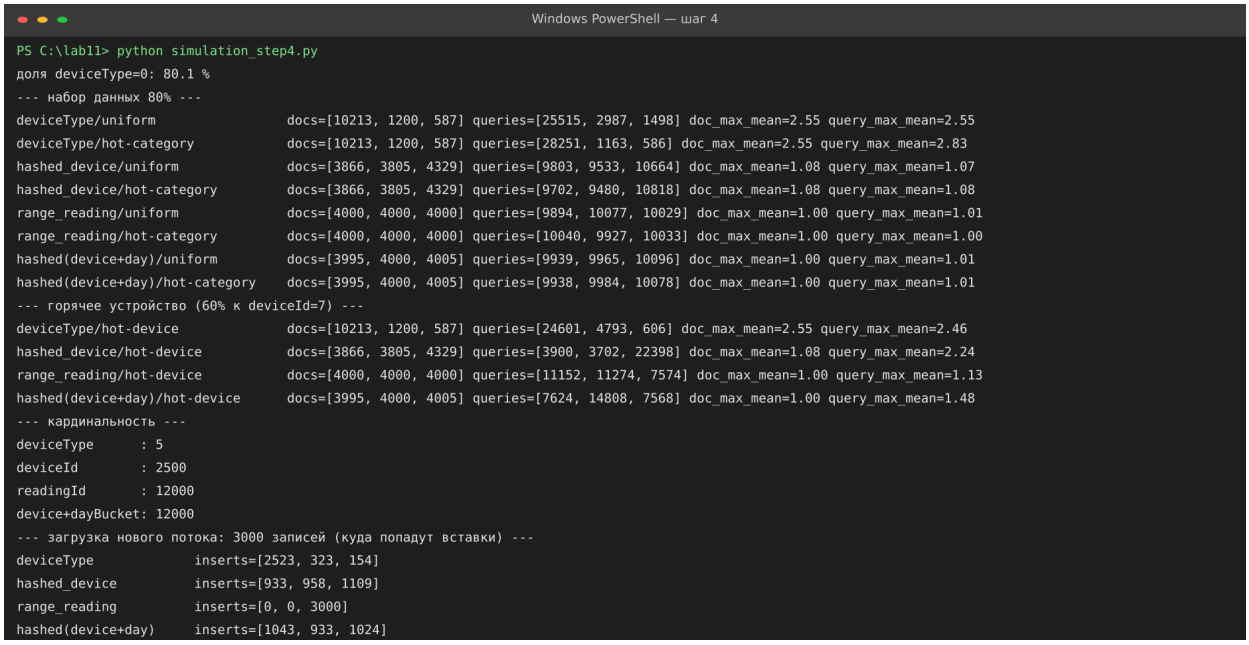


Рисунок 6. Скриншот: запуск simulation_step4.py.

Сравнение двух стратегий: **(1) hashed deviceId** и **(2) deviceId + dayBucket**. Аргументы выбора:

- **Горячие устройства.** При hashed deviceId все документы одного устройства имеют одно значение ключа и попадают на один шард: в эксперименте шард C получил 74,7% запросов (max/mean 2,24). С парой deviceId + dayBucket данные устройства распределены по нескольким шардам, и перекос снизился до 1,48. Полностью он не исчезает: у устройства лишь 5 различных пар (5 суток), поэтому нужны более мелкие бакеты, кеширование или реплики для чтения.
- **Рост данных одного устройства.** В IoT показания накапливаются бесконечно. Все документы с одним значением ключа нельзя разделить на разные чанки (риск «jumbo chunk»). У пары с dayBucket кардинальность растёт со временем (12 000 против 2500), поэтому данные делятся мельче.
- **Цена целевых запросов.** AP-01 (45%) на hashed deviceId всегда targeted. Для составного ключа он остаётся targeted только при указании dayBucket (или при ranged-ключе с префиксом deviceId); иначе затрагиваются несколько шардов (в модели до 3). Это компромисс «равномерность ↔ простота маршрутизации».

- **Запись.** Оба варианта распределяют вставки равномерно (табл. 4), в отличие от монотонного readingId (0/0/3000) и низкокардинального deviceType (84% в шард A).

Решение: для текущего профиля (AP-01 и AP-04 — 75% нагрузки) достаточно hashed deviceId; при появлении «шумных» устройств и росте истории рекомендуется перейти на составной ключ deviceId + dayBucket.

8. Дополнительное задание (реальный sharded cluster)

Учебный шардированный кластер с правами администратора не предоставлялся, поэтому этот раздел **не выполнялся**; одиночный mongod не является шардированным кластером. Для варианта 09 команды (выполнять только на изолированном учебном стенде через mongos) выглядели бы так:

```
sh.status()
use university_lab
db.readings.createIndex({deviceId: "hashed"})
sh.shardCollection("university_lab.readings", {deviceId: "hashed"})
db.readings.getShardDistribution()
db.readings.find({deviceId: 1001}).explain("executionStats")   // ожидаем targeted: один шард в плане
```

Результаты реального стенда не следует ожидать идентичными модели: MongoDB использует собственное разбиение на чанки и балансировщик.

9. Заключение

Рекомендуемый shard key: hashed `deviceId` — для текущего профиля нагрузки; при росте истории и появлении «горячих» устройств — составной ключ `deviceId + dayBucket`. Ключ deviceType отвергнут: 5 значений, max/mean по запросам 2,28 при 60% нагрузке на один тип и 2,55 при 80% данных в одном типе. Ключ `readingId` отвергнут для записи: монотонность направляет весь новый поток (3000 из 3000) в один шард, хотя он удобен для диапазонных запросов AP-02.

Компромиссы: hashed-ключ даёт равномерность, но делает диапазонные запросы scatter-gather; составной ключ лучше против hot key, но требует указывать оба поля в фильтре.

Риски: горячее устройство (2,24 при hashed deviceId); jumbo chunks при неограниченном росте данных одного значения ключа; дорогие scatter-gather для AP-02/03/06; смена shard key в рабочей системе — сложная операция, поэтому ключ важно проверить заранее.

Ограничения модели: распределение по модулю и SHA-256 не воспроизводят чанки и балансировщик MongoDB; нет сети, репликации и миграций; данные статичны, вставки смоделированы отдельным экспериментом; запросы считаются равнозначными по стоимости; результаты зависят от seed и синтетических распределений.

10. Ответы на контрольные вопросы

1. Шардирование делит данные между узлами (каждый хранит часть), репликация создаёт копии одних и тех же данных ради отказоустойчивости.
2. Shard key — поле(я), по которым определяются размещение документа и маршрутизация запросов. От него зависят баланс, доля targeted-запросов и возможность масштабирования; изменить его позже трудно.
3. Кардинальность — число различных значений ключа. При низкой (у deviceType — 5) данные делятся на крупные неделимые группы, шардов может быть «больше, чем значений», и возникает перекос.
4. Ranged удобнее для диапазонных запросов и при необходимости локальности данных (AP-02), если ключ не монотонен или запись не критична; hashed — для равномерной записи.
5. Новые значения монотонного ключа всегда попадают в последний диапазон, поэтому все вставки идут на один шард (в опыте 3000 из 3000).
6. Targeted — запрос с shard key (префиксом) идёт на один/несколько нужных шардов; scatter-gather — рассылка на все шарды и объединение ответов.
7. Размещение зависит от значений ключа в документах, а нагрузка — от популярности значений в запросах: при hot 60% документы 40/40/20, а запросы 76/16/8%.
8. Max/mean — во сколько раз самый загруженный шард нагружен сильнее среднего: 1,0 — идеал, 2,28 — шард A в 2,28 раза выше среднего.
9. Модель не учитывает чанки, балансировщик, сеть, репликацию, размеры документов и индексы; SHA-256 и модуль — упрощения.
10. Access Patterns и их частоты, кардинальность и частоты значений ключа, рост данных, долю targeted/scatter-gather запросов, пиковую нагрузку, долю hot-ключей и размер данных.